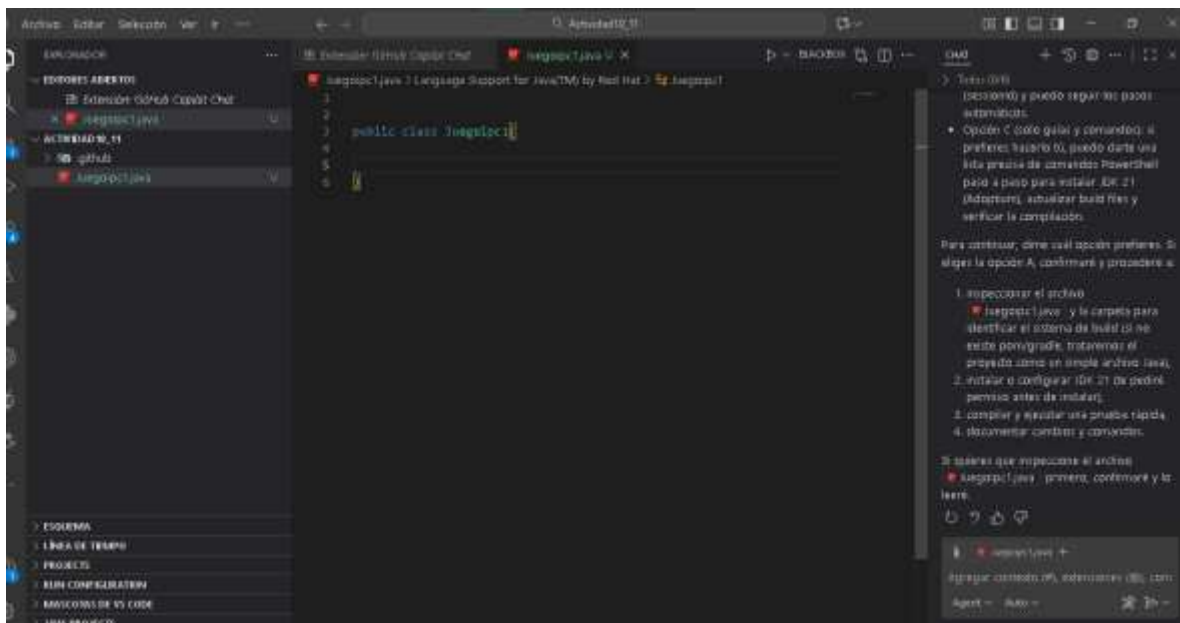
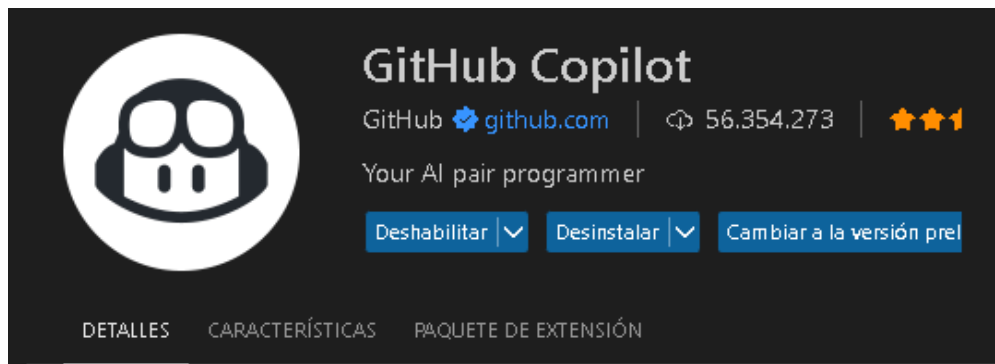
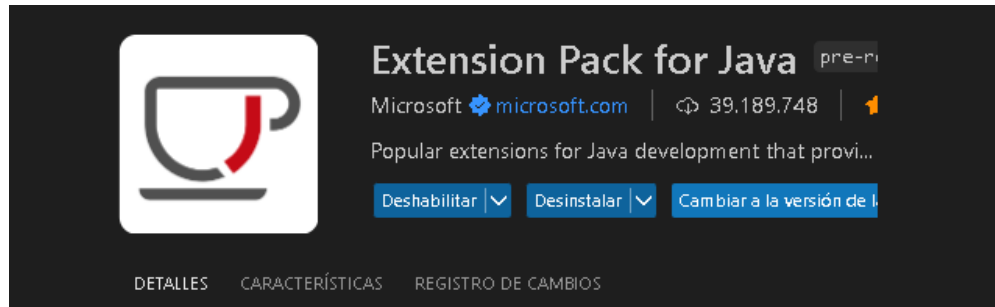


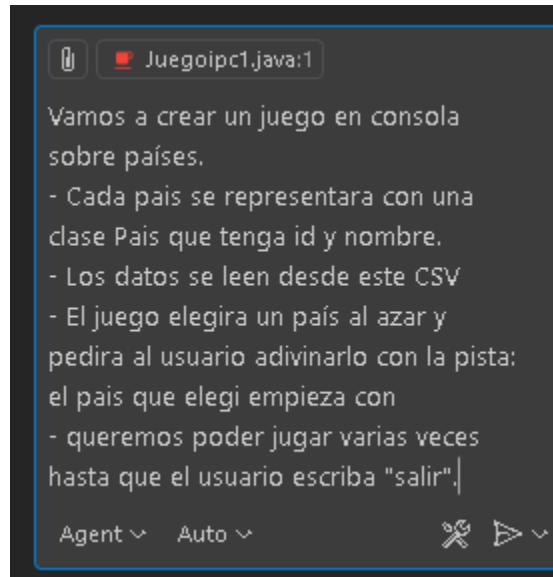
Actividad 10-11

Inicialización de la Actividad 10-11 Con Visual Studio Code con la Extensión Pack Java, GitHub Copilot y GitHub Copilot Chat.



Indicamos con el GitHub Copilot Chat.

Le damos el contexto del Juego para que nos apoye con la base de dicho Juego.



Luego, le pedimos que nos proporcionara solo la base para empezar la actividad, para poder guiarnos mejor.

El resultado fue que nos creó la clase Pais para poder utilizarla, y además nos dio los setters y getters.

También nos creó la base para poder iniciar el juego en un Main, incluyendo los recursos necesarios como el ArrayList<>, Scanner y Random, ya instanciados, para poder implementarlos directamente.

Incluso nos dio parte del ciclo del juego y los textos básicos para interactuar, y nos proporcionó dos funciones: cargarPaises (); y jugarRonda (); con funcionamiento distinto, que serían llamadas desde el Main.

```

import java.util.ArrayList;
import java.util.Random;
import java.util.Scanner;

class Pais {
    private String id;
    private String nombre;

    public Pais(String id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }

    public String getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }
}

```

```

public class Juegoipc1 {
    private static ArrayList<Pais> paises = new ArrayList<>();
    private static Scanner scanner = new Scanner(System.in);
    private static Random random = new Random();

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        // TODO: Implementar la carga del archivo CSV
        cargarPaises();

        String respuesta;
        do {
            jugarRonda();
            System.out.println(x:"\n¿Quieres seguir jugando? (Escribe 'salir' para t
            respuesta = scanner.nextLine());
        } while (!respuesta.equalsIgnoreCase(anotherString:"salir"));

        System.out.println(x:"¡Gracias por jugar!");
        scanner.close();
    }
}

```

Me crea la base para poder iniciarlo en un Main, y los recursos necesario como el ArrayList<> , Scanner y Random de una manera ya instanciada y para poder implementarlo, además me da la parte del ciclo y el texto para poder jugar,

además de dos funciones `CargarPaises()`; y `jugarRonda()`; ambas con funcionamiento distintos y que serán llamadas.

Colocamos la lógica para cargar los países al `ArrayList` y así poder empezar el juego con información real.

```
private static void cargarPaises() {  
    // TODO: Implementar la lectura del archivo CSV  
    // Ejemplo de estructura:  
    // try (BufferedReader br = new BufferedReader(new FileReader("ruta_al_archi  
    //     String linea;  
    //     while ((linea = br.readLine()) != null) {  
    //         String[] datos = linea.split(",");  
    //         paises.add(new Pais(datos[0], datos[1]));  
    //     }  
    // } catch (Exception e) {  
    //     System.out.println("Error al cargar el archivo: " + e.getMessage());  
    // }  
}
```

Ahora la función `Cargar países` me lo da con comentarios, ya definidos del funcionamiento de cargar archivo, aquí solo vamos a modificar según como nos convenga y no tomar todo literalmente del programa, según lo que necesitemos.

```

private static void cargarPaíses() {
    // Implementar carga de países desde un archivo csv

    try(BufferedReader leer = new BufferedReader(new FileReader(fileName:"data.
    csv"))){

        String linea;

        while((linea = leer.readLine()) != null){
            String[] datos = linea.split(regex:",");
            String pais = datos[0];
            String pista = datos[1];

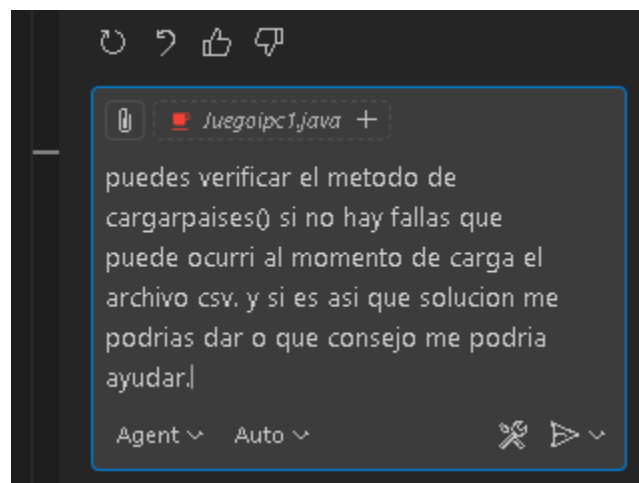
            //verificamos que sean solo dos columnas

            if (datos.length != 2) {
                System.out.println(x:"Formato incorrecto para Carga Csv");
            }else{
                Pais pais_nuevo = new Pais(pais, pista);
                paises.add(pais_nuevo); //agregamos los países al arraylist
            }
        }

    }catch(Exception e){
        System.out.println(x:"Error al cargar el Archivo");
    }
}

```

Colocamos la lógica para cargar los países al array y así empezar el juego con información.



Le preguntamos si considera que este bien la lógica para así poder cargar archivos csv, y si no fuera el caso que me pudiera brindar la lógica adecuada o algún consejo de mejoramiento para que el funcionamiento de ese método se exitoso.

Resultados de su mejoramiento Notorio :3

```
// Lectura mínima robusta: UTF-8, skip posible encabezado y validación simple
String path = "data.csv";
int lineNum = 0;

try (BufferedReader leer = new BufferedReader(
    new InputStreamReader(new FileInputStream(path), StandardCharsets.UTF_8))) {

    String linea;
    while ((linea = leer.readLine()) != null) {
        lineNum++;
        linea = linea.trim();
        if (linea.isEmpty()) {
            continue; // saltar líneas vacías
        }

        // detectar y saltar encabezado simple
        if (lineNum == 1) {
            String low = linea.toLowerCase();
            if (low.contains(":nombre") || low.contains(":id") || low.contains(":pais")) {
                continue;
            }
        }

        String[] datos = linea.split(regex(",");
        if (datos.length < 2) {
            System.out.println("Línea " + lineNum + " ignorada (formato
```

```

        continue;
    }

    String id = datos[0].trim().replaceAll(regex:"^\"|\"$",
replacement:"");
    String nombre = datos[1].trim().replaceAll(regex:"^\"|\"$",
replacement:"");

    if (id.isEmpty() || nombre.isEmpty()) {
        System.out.println("Línea " + lineNum + " con campos vacíos,
ignorada.");
        continue;
    }

    paises.add(new Pais(id, nombre));
}

System.out.println("Países cargados: " + paises.size());

} catch (FileNotFoundException e) {
    System.out.println("Archivo no encontrado: " + path + ". Coloca 'data.
csv' en la carpeta del proyecto o modifica la ruta.");
} catch (IOException e) {
    System.out.println("Error leyendo archivo CSV: " + e.getMessage());
}
}

```

Qué cambié (breve)

- `cargarPaises()` ahora:
 - Lee `data.csv` con UTF-8.
 - Salta una posible primera línea de encabezado si contiene "nombre", "id" o "pais".
 - Ignora líneas vacías y líneas con menos de 2 columnas (muestra aviso con número de línea).
 - Elimina comillas envolventes y valida campos no vacíos antes de añadir al ArrayList.
 - Maneja `FileNotFoundException` y errores de I/O con mensajes informativos.
- Importaciones actualizadas para soportar la lectura UTF-8.

Razones por la que modifíco dicho código.

Estado de verificación

- Hice una comprobación rápida de errores; solo quedan advertencias sobre campos no usados (p. ej. la lista `pista`) y sugerencias de convertir campos en `final`. No hay errores de sintaxis críticos después del cambio.

Sugerencias pequeñas (opcional, no obligatorio)

- Si tu CSV puede contener comas dentro de campos citados, usar una librería como OpenCSV o Apache Commons CSV es más seguro.
- Si quieres, puedo:
 - Permitir pasar la ruta del CSV como argumento a `main` (rápido).
 - Eliminar la variable `pista` si no la usarás (quita la advertencia).
 - Implementar la verificación de la respuesta del usuario en `jugarRonda()` (comprobación de adivinanza).

Ahora implementamos la lógica del juego según mi propio criterio para cumplir los requerimientos dados.


```
//ahora vmaos a implementar la logica del juego
do {

    System.out.println(x:"¿Cual crees que el pais?");
    String intento = scanner.nextLine();

    if (paisElegido.getNombre().equalsIgnoreCase(intento)) {
        System.out.println("Correcto ¡Has adivinado el pais!" + ":"+paisElegido.
            getNombre() + " " + "Lo lograste en el intento Numero" + " "
            +cantidad_intentos);
        acierto = true;
        break;
    }else{
        System.out.println(x:"Incorrecto, intenta de neuvo.");
        cantidad_intentos++;
    }
} while (acierto != false);
```

Le pregunte a Chat de Git que deficiencia del Juego tiene la lógica, para saber una opinión y conocer que posibles mejoramientos debería de implementar.

ya hice segun mi opion la logica del Juego, Jugar Ronda de lo que me pidieron dar una pista al usuario y que el trate de adivinar, ahora necesito que mires la deficiencia de esa parte del codigo?

Mejoramiento hecho por la IA:

```

private static String normalize(String s) {
    if (s == null) return "";
    String n = s.trim().toLowerCase();
    n = java.text.Normalizer.normalize(n, java.text.Normalizer.Form.NFD)
        .replaceAll("\\p{InCombiningDiacriticalMarks}+", "");
    return n;
}

private static void jugarRonda() {
    if (países.isEmpty()) {
        System.out.println("No hay países cargados en el juego.");
        return;
    }

    País paísElegido = países.get(random.nextInt(países.size()));
    String nombreRaw = paísElegido.getNombre();
    if (nombreRaw == null) nombreRaw = "";
    String nombre = nombreRaw.trim();
    if (nombre.isEmpty()) {
        System.out.println("El país seleccionado no tiene nombre válido, se omite esta vez.");
        return;
    }
}

```

```

char primera = nombre.charAt(0);
System.out.println("El país que elegí empieza con: " + Character.toUpperCase(primera));
System.out.println("Escribe tu respuesta (o 'salir' para terminar esta partida, 'pasar' para continuar): ");

String intento;
String nombreNorm = normalize(nombre);
while (true) {
    intento = scanner.nextLine();
    if (intento == null) continue;
    intento = intento.trim();
    if (intento.equalsIgnoreCase("salir")) {
        // Dejar que el bucle principal detecte salir; aquí solo terminamos la ronda
        System.out.println("Has salido de la ronda.");
        break;
    }
    if (intento.equalsIgnoreCase("pasar")) {
        System.out.println("Respuesta correcta: " + nombre);
        break;
    }

    String intentoNorm = normalize(intento);
    if (intentoNorm.equals(nombreNorm)) {
        System.out.println("¡Correcto! El país era: " + nombre);
        break;
    }
}

```

```

String intentoNorm = normalize(intento);
if (intentoNorm.equals(nombreNorm)) {
    System.out.println("¡Correcto! El país era: " + nombre);
    break;
} else {
    System.out.println("Incorrecto. Intenta otra vez (o escribe 'pasar' / 'salir')");
}
}

```

Hay mejoras que puede ayudar mucho al manejo de errores como cuando el usuario no ingresar los datos en el Scanner y le da Enter ocasionado que no envíe ninguna información para comparar, también uno de los detalles más importantes son los espacios que el Usuario puede enviar y evitarlos con un `.trim()` e incluso de

interacción con el Usuario como la opcion pasar que es un detalle muy importante y que es una opcion para que el Usuario pueda seguir jugando sin ser presionado a responder, una manera sutil para el funcionamiento.

Codigo del Juego:

```
private static void jugarRonda() {  
    boolean acierto = false;  
    int cantidad_intentos = 0;  
  
    if (países.isEmpty()) { //si esta vacío el arraylist mostramos no hay países  
        cargados  
        System.out.println(x:"No hay países cargados en el juego.");  
        return;  
    }  
  
    //implementamos la logica del juego  
    País paísElegido = países.get(random.nextInt(países.size()));  
    System.out.println("El país que elegí empieza con: " + paísElegido.getNombre().  
        substring(beginIndex:0,endIndex:2));  
  
    //ahora vamos a implementar la logica del juego  
  
    while(true){  
        System.out.println(x:"¿Cual crees que el país? (Si no lo sabe escriba pasar/  
            salir)");  
  
        String intento = scanner.nextLine();
```

```
//consejo hecho por la IA

if (intento.isEmpty()) {
    System.out.println(x:"Entrada invalida, Dejo espacio en Blanco");
    continue;
}

if (intento.equalsIgnoreCase(anotherString:"Salir")) {
    System.out.println(x:"Has decidido salir del juego !!");
    break;
}

intento = intento.trim();

if (paisElegido.getNombre().equalsIgnoreCase(intento)) {
    System.out.println("Correcto ¡Has adivinado el pais!" + ":" + paisElegido.getNombre() + " " + "Lo lograste en el intento Numero" + " " + cantidad_intentos);
    acierto = true;
    break;
} else if (intento.equalsIgnoreCase(anotherString:"Pasar")) {
    System.out.println("Has decidido pasar. El pais era:" + paisElegido.getNombre());
    break;
} else {
    System.out.println(x:"Incorrecto, intenta de nuevo.");
    cantidad_intentos++;
}
```

Ahora el funcionamiento:

Funcionamiento del juego, el programa da una pista del país que debes adivinar (en nuestro caso, usando el código del país).

Si falla, te pide que intentes de nuevo.

Puedes escribir “pasar” para ver la respuesta y pasar a otro país, o “salir” para terminar el juego.

Así seguimos jugando hasta que el usuario quiera parar.

```
El país que elegí tiene este código: IT
¿Cual crees que el pais? (Si no lo sabe escriba pasar/salir)
```

Aquí el programa nos da una pista del país que debemos adivinar, aquí una prueba de que fallamos y nos pide que intentemos de nuevo, y otras opciones de si queremos pasar a otro país o salir de la oportunidad del juego mas no del programa.

```
El país que elegí tiene este código: IT
¿Cual crees que el pais? (Si no lo sabe escriba pasar/salir)
pasar
Has decidido pasar. El país era:Italy
```

Aquí pasamos a otro porque el Usuario no pudo adivinar y de una el programa después de pasarlo nos da la respuesta.

Ahora jugaremos para ver a través de los codigos podemos adivinar el pais y asi seguir hasta que el usuario se canse o quiera salir.

```
jdt_ws\x5cActividad10_11_35ee2e3b\x5cbin' 'Juegoipci' ;260fdf06-de5a-4866-a069-74f85aa45306Países cargados
250
El país que elegí tiene este código: MT
¿Cual crees que el pais? (Si no lo sabe escriba pasar/salir)
Montenegro
Incorrecto, intenta de nuevo.
¿Cual crees que el pais? (Si no lo sabe escriba pasar/salir)
pasar
Has decidido pasar. El país era:Malta

¿Quieres seguir jugando? (Escribe 'salir' para terminar)
si
El país que elegí tiene este código: KH
¿Cual crees que el pais? (Si no lo sabe escriba pasar/salir)
pasar
Has decidido pasar. El país era:Cambodia

¿Quieres seguir jugando? (Escribe 'salir' para terminar)
no
El país que elegí tiene este código: CX
¿Cual crees que el pais? (Si no lo sabe escriba pasar/salir)
Christmas Island
Correcto ¡Has adivinado el país!:Christmas Island Lo lograste en el intento Numero 0

¿Quieres seguir jugando? (Escribe 'salir' para terminar)
salir
¡Gracias por jugar!
```

Y aquí el codigo funcionando.

47	Christmas Island	
48	Christmas Island, CX	

Y ahí verificando el codigo que se muestra en el archivo.csv

Decisiones que tomamos al usar el código de la IA

Bueno al utilizar el código de que nos proporciono no todo era como queríamos porque no tenía el contexto completo entonces por eso pedimos solo la base o plantilla para que tuviéramos como empezar y así tener una idea de como desarrollar el juego y así irnos por partes que nos servían y que cosas habían de cambiar. Por ejemplo, la IA nos dio la base del juego con la clase Pais, y los métodos Cargar () y JugarRonda () pero algunas cosas no encajaban del todo con lo que pedía la actividad.

Entonces decidimos cambiar partes del código como los métodos datos como en la forma en que se cargaban los países para que leyera bien el archivo csv y guardara el nombre del país y el código correctamente en el ArrayList<>. También cambiamos las pistas del juego porque queríamos que el código del país fuera la pista y no solamente la primera letra del nombre como la IA había puesto.

Otra de las decisiones fue mejorar un poco la interacción con el usuario porque si uno solo presionaba enter o dejaba espacio en blanco el programa no tiraba algún error o algo que notificara al usuario sobre la acción. Así que le agregamos validaciones y la opción de pasar para que el juego no se quedara trabado y así darle un giro algo mas positivo para el usuario.

También decidimos mantener un estilo de juego mas simple en consola sin hacerlo tan complicado solo que pedía la actividad y que funcionara bien, como resumen usamos lo que nos sirvió del código generado por la IA y le metimos nuestros propios cambios para que quedara mas a nuestro gusto y mas funcional y solo tomamos lo que consideramos que fuera de ayuda.